

Université Technologique de Compiegne

Génie Informatique

Rapport TZ

Visualisation et analyse du réseau de transport urbain de Toronto



Encadrants :

ALAYA Elmokhtar
BOUZEBDA Salim

TZ associé :

Alexandre Bidaux

Étudiant :

Clément Martins

Semestre :

GI5 – Automne 2025

Année universitaire 2025–2026

1 Introduction

Ce projet vise à concevoir une application permettant de visualiser l'état actuel du réseau de transport urbain, de consulter les conditions météorologiques en temps réel, ainsi que d'analyser des statistiques globales et spécifiques à chaque ligne. L'interface principale repose sur une carte interactive affichant les lignes, les arrêts et des informations dynamiques liées au trafic et aux perturbations.

L'objectif est d'offrir une vision claire, synthétique et contextualisée du réseau afin d'aider les usagers à anticiper leurs déplacements.

Ce projet a été réalisé en étroite collaboration avec **Alexandre Bidaux** qui a modélisé et entraîné le système prédictif.

1.1 Objectifs fonctionnels

L'application doit permettre à l'utilisateur de :

- Consulter la météo actuelle sur la zone géographique concernée, incluant les conditions, la température et un indicateur visuel représentatif.
- Visualiser les lignes de bus sur une carte interactive, accompagnées de l'ensemble des arrêts et d'un indicateur d'état (trafic fluide, perturbé ou interrompu).
- Accéder aux informations détaillées d'une ligne, telles que les incidents en cours, les incidents historiques, ainsi que des prévisions de perturbations lorsqu'elles sont disponibles.
- Analyser des statistiques globales du réseau, mettant en évidence les lignes les plus impactées, le niveau global de perturbation et les tendances temporelles.

1.2 Définition des fonctionnalités et de leurs priorités

Fonctionnalité	Priorité
Carte interactive des lignes et arrêts	1
Prédiction de retards	2
Statistiques réseau et par ligne	3
Météo actuelle	4
État du trafic et incidents	5

2 Diagramme de cas d'utilisation

Pour représenter les interactions entre les utilisateurs et le système, un diagramme de cas d'utilisation a été élaboré. Ce diagramme illustre les principales fonctionnalités accessibles aux utilisateurs, telles que la consultation de la météo, la visualisation des lignes de bus, l'accès aux détails des lignes et l'analyse des statistiques.

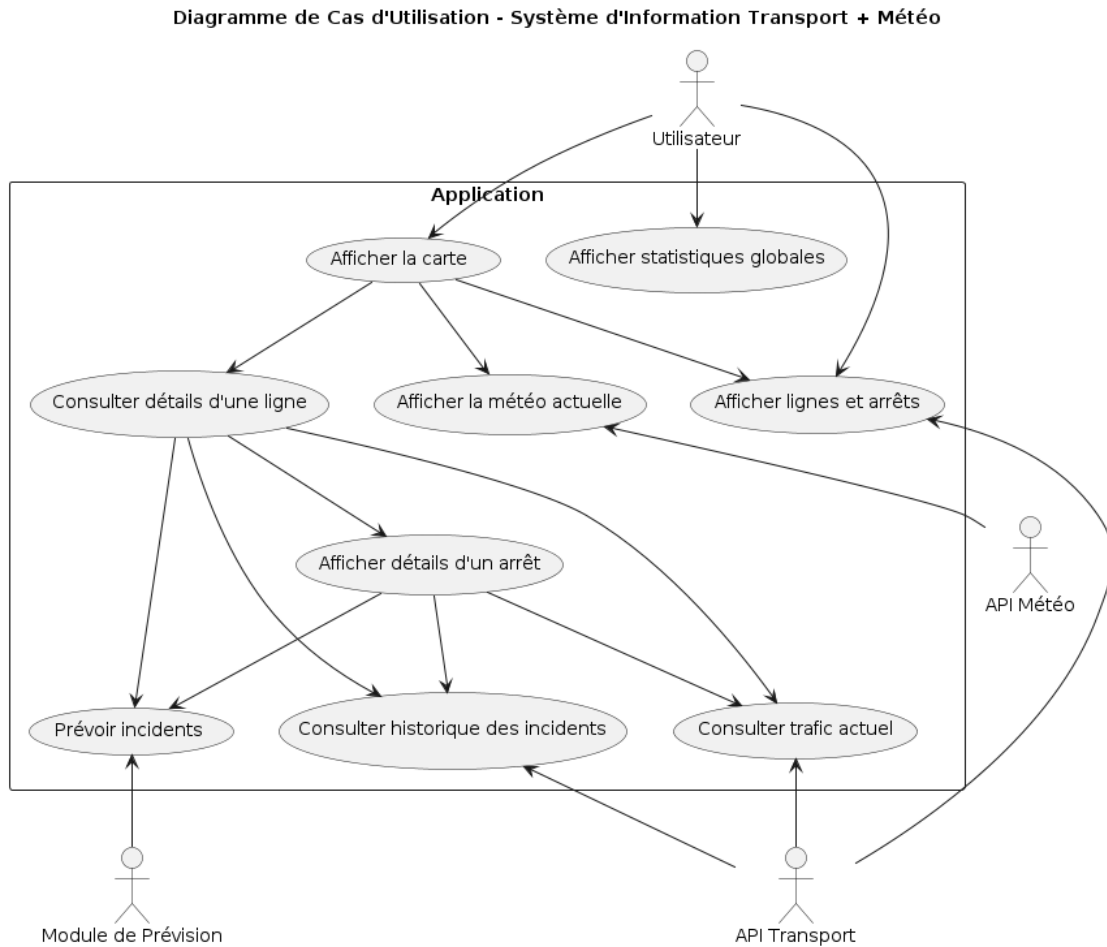


FIGURE 1 – Diagramme de cas d'utilisation

3 Diagramme de classes

Le diagramme de classes ci-dessous présente la structure de notre back-end, mettant en évidence les principales entités, leurs attributs et les relations entre elles. Ce diagramme est essentiel pour comprendre comment les données sont organisées et gérées au sein de l'application.

Il constituera la base de nos appels API et de la gestion des données. Cette partie est cruciale pour assurer une architecture solide et évolutive de notre application. Elle a été discutée et validée avec Alexandre Bidaux avant de commencer le développement afin de se mettre d'accord sur la communication entre nos deux TX.

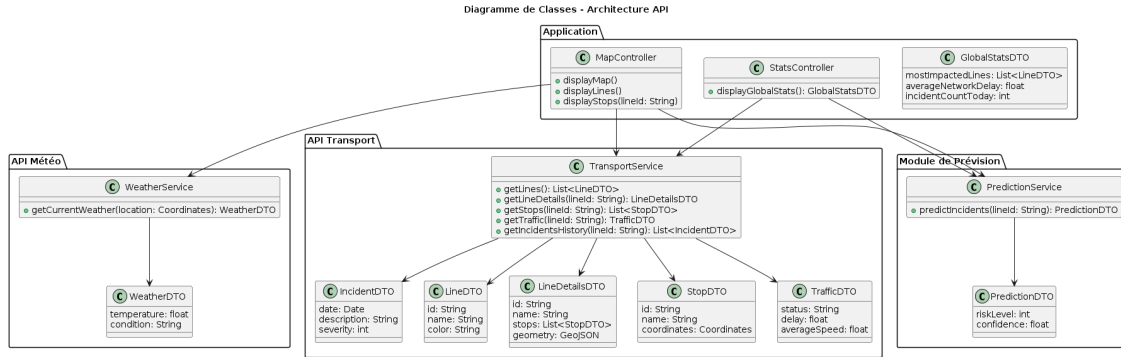


FIGURE 2 – Diagramme de classes

On peut ainsi générer à partir de ce schéma notre API RESTful avec les routes suivantes :

<https://editor.swagger.io>

avec le fichier : [APIRET.yaml](#)

4 Proof of Concept

Avant de s'engager dans la mise en œuvre complète du système, un Proof of Concept (PoC) a été réalisé afin de vérifier la cohérence des choix techniques et la faisabilité des fonctionnalités clés. L'objectif n'était pas de proposer une version exhaustive de l'application, mais de valider étape par étape les principaux éléments qui constituent son cœur fonctionnel.

Le PoC a tout d'abord permis de tester l'intégration d'une carte interactive basée sur **Mapbox GL JS**, affichée au sein d'une application front-end développée avec **React**. Nous avons vérifié la possibilité de représenter dynamiquement des lignes de bus sous forme de tracés géographiques, ainsi que d'ajouter des points d'intérêt tels que les arrêts. Ce premier jalon a servi à confirmer la compatibilité entre les données de coordonnées, la logique de rendu en temps réel et les contraintes de performance liées au navigateur.

De plus, Mapbox GL offre une grande flexibilité en termes de personnalisation visuelle, ce qui a permis d'adapter l'apparence de la carte aux besoins spécifiques du projet, notamment en termes de styles et de couches d'information supplémentaires. Il est aussi possible d'intégrer **des effets visuels avancés** (pluie et neige) sur la carte et d'y inclure **Three.js** pour, à terme, intégrer une visualisation en temps réel des bus en 3D.

La carte personnalisée est accessible ici :

[Mapbox carte](#)

Elle a été réalisée de façon à faire ressortir les routes et à mettre en avant les lignes de bus.

Parallèlement, une première couche de back-end a été mise en place à l'aide de Node.js et Express, structurée de manière à évoluer vers une architecture en microservices. Une attention particulière a été portée à l'isolation des clés d'API et des paramètres sensibles via un fichier `.env`, afin de garantir une configuration modulaire et sécurisée.

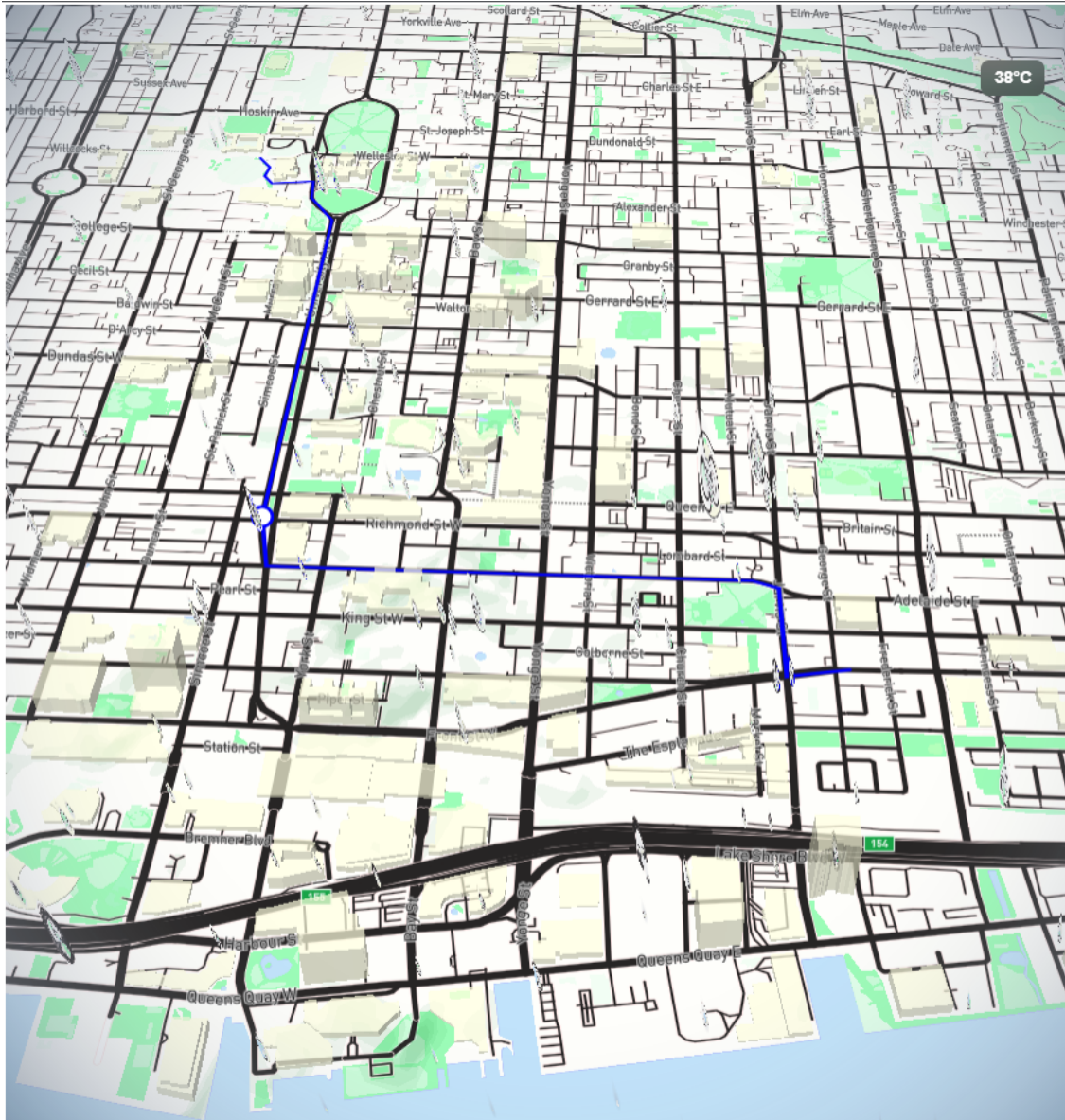


FIGURE 3 – Proof of Concept

5 Architecture générale du système

Le système repose sur une architecture en trois couches distinctes et complémentaires, permettant une séparation claire des responsabilités et facilitant la maintenance ainsi que l'évolutivité de l'application.

5.1 Vue d'ensemble de l'architecture

L'architecture globale peut être décomposée en trois composantes principales :

1. **Frontend (React + Mapbox GL)** : interface utilisateur interactive permettant la visualisation cartographique des lignes de bus, des arrêts et des statistiques associées. Le frontend

communiquent exclusivement avec le backend via des endpoints REST et ne sollicitent jamais directement les services externes.

2. **Backend (Node.js + Express + TypeScript)** : couche applicative centrale orchestrant l'ensemble des services. Elle expose une API REST structurée, gère la logique métier, effectue les appels aux services externes (prédiction, météo) et assure la mise en cache des données pour optimiser les performances. Le backend sert également les données GTFS prétraitées sous forme de JSON normalisé.
3. **Service de prédiction (Python + Flask + XGBoost)** : microservice autonome dédié à l'inférence de retards potentiels. Il encapsule le modèle de Machine Learning entraîné et expose un endpoint de prédiction indépendant, permettant ainsi de déployer et de scaler ce service sans impacter le reste de l'application.

5.2 Flux de données

Le schéma de communication entre les différentes couches suit un modèle de requête-réponse synchrone :

- l'utilisateur interagit avec l'interface graphique (sélection de lignes, consultation des statistiques) ;
- le frontend transmet les requêtes au backend via HTTP ;
- le backend interroge le service de prédiction Python lorsqu'une estimation de retard est demandée ;
- le backend récupère également les données météorologiques actuelles depuis l'API Meteo-source, avec mise en cache pour limiter les appels répétés ;
- les réponses remontent jusqu'à l'interface utilisateur, où elles sont présentées de manière claire et structurée.

5.3 Avantages de cette approche

Cette séparation en couches offre plusieurs avantages notables :

- **Modularité** : chaque composant peut évoluer indépendamment.
- **Scalabilité** : les services peuvent être répliqués ou déployés sur des infrastructures distinctes selon les besoins en charge.
- **Maintenabilité** : les responsabilités étant claires, la maintenance et le débogage sont facilités.
- **Testabilité** : chaque couche peut être testée unitairement sans dépendance forte aux autres.

6 Création des routes avec les données GTFS

La construction des routes exploitables pour la visualisation cartographique constitue l'une des étapes les plus critiques du projet. Elle repose sur l'exploitation des données de transport au format GTFS (General Transit Feed Specification), standard international largement adopté par les agences de transport urbain.

6.1 Présentation du format GTFS

Le format GTFS est un ensemble de fichiers texte structurés au format CSV, décrivant les lignes, les arrêts, les horaires et les itinéraires des transports en commun. Chaque fichier contient des informations spécifiques, liées entre elles par des identifiants uniques.

Les principaux fichiers exploités dans ce projet sont :

- **routes.txt** : décrit les lignes de transport. Chaque ligne possède un identifiant unique (`route_id`), un nom court (`route_short_name`), un nom long (`route_long_name`), une couleur associée (`route_color`) et un type de transport (`route_type`). Pour les bus, le type est 3.
- **trips.txt** : définit les trajets (*trips*) effectués par les véhicules. Un trip représente un parcours spécifique d'une ligne, à un moment donné. Ce fichier contient notamment `trip_id`, `route_id`, `shape_id` (référence à la géométrie du trajet) et `direction_id` (aller/retour).
- **stop_times.txt** : associe chaque trip à une séquence d'arrêts ordonnés. Les colonnes principales sont `trip_id`, `stop_id` et `stop_sequence`. Ce fichier permet de reconstituer l'ordre des arrêts pour chaque trajet.
- **stops.txt** : contient les informations géographiques et descriptives des arrêts (`stop_id`, `stop_name`, `stop_lat`, `stop_lon`).
- **shapes.txt** (optionnel) : fournit la géométrie précise des itinéraires sous forme de séquences de points (`shape_id`, `shape_pt_lat`, `shape_pt_lon`, `shape_pt_sequence`).
- **calendar.txt** / **calendar_dates.txt** : définissent les jours de service pour chaque trip (non utilisés directement dans notre pipeline de construction de routes, mais essentiels pour les horaires).

6.2 Pipeline complet de construction des routes

Le processus de construction des routes s'articule en plusieurs étapes successives, réalisées principalement dans le script Python `BusStops.py`.

6.2.1 Étape 1 : filtrage des lignes autorisées

Afin de limiter le volume de données et de ne traiter que les lignes pertinentes (par exemple, celles effectivement desservies par l'agence TTC à Toronto), un fichier de configuration `lignes_bus.json` liste les identifiants de routes autorisés. Ces lignes correspondent aux lignes pouvant être prédites par notre système prédictif.

```
1 allowed_route_ids = load_allowed_route_ids("../res/extract/lignes_bus.json")
```

6.2.2 Étape 2 : lecture et enrichissement des métadonnées de lignes

Le fichier `routes.txt` est lu ligne par ligne. Pour chaque ligne autorisée, les informations descriptives (nom, couleur, nom long) sont extraites et stockées.

```
1 with routes_path.open("r", encoding="utf-8-sig", newline="") as f:
2     r = csv.DictReader(f)
3     for row in r:
4         route_id = row.get("route_id").strip()
5         if route_id not in allowed_route_ids:
6             continue
7         route_info[route_id] = {
8             "id": int(route_id),
9             "nom": row.get("route_short_name").strip(),
10            "long_nom": row.get("route_long_name").strip(),
11            "couleur": row.get("route_color").strip(),
12        }
```


6.2.3 Étape 3 : canonicalisation des arrêts

Un problème récurrent dans les données GTFS est la duplication d'arrêts physiquement identiques mais possédant des identifiants distincts. Pour corriger ce problème, une étape de canonicalisation est appliquée : tous les arrêts partageant les mêmes coordonnées GPS (latitude/longitude) sont regroupés sous un identifiant canonique unique.

```

1 coord_to_ids: dict[tuple[float, float], list[str]] = defaultdict(list)
2
3 with stops_path.open("r", encoding="utf-8-sig", newline="") as f:
4     r = csv.DictReader(f)
5     for row in r:
6         stop_id = row.get("stop_id").strip()
7         lat = float(row.get("stop_lat").strip())
8         lon = float(row.get("stop_lon").strip())
9         coord_to_ids[(lat, lon)].append(stop_id)
10
11 coord_to_canon = {c: sorted(ids, key=stop_sort_key)[0] for c, ids in
12     coord_to_ids.items()}
13 stop_to_canon = {sid: coord_to_canon[(lat, lon)] for sid, (lat, lon) in
14     stop_coords.items()}

```

6.2.4 Étape 4 : association des trips aux lignes

Le fichier trips.txt est parcouru pour identifier tous les trips associés à chaque ligne et récupérer les shape_id éventuels.

```

1 route_to_trips: dict[str, set[str]] = defaultdict(set)
2 trip_to_shape: dict[str, str] = {}
3
4 with trips_path.open("r", encoding="utf-8-sig", newline="") as f:
5     r = csv.DictReader(f)
6     for row in r:
7         route_id = row.get("route_id").strip()
8         if route_id not in route_ids:
9             continue
10        trip_id = row.get("trip_id").strip()
11        route_to_trips[route_id].add(trip_id)
12
13        shape_id = row.get("shape_id").strip()
14        if shape_id:
15            trip_to_shape[trip_id] = shape_id

```

6.2.5 Étape 5 : reconstitution de l'ordre des arrêts par trip

stop_times.txt fournit pour chaque trip la liste des arrêts et l'ordre de passage (stop_sequence). Les arrêts sont convertis en identifiants canoniques.

```

1 trip_to_seq: dict[str, list[tuple[int, str]]] = defaultdict(list)
2
3 with stop_times_path.open("r", encoding="utf-8-sig", newline="") as f:
4     r = csv.DictReader(f)
5     for row in r:

```



```

6     trip_id = row.get("trip_id").strip()
7     if trip_id not in trip_to_route:
8         continue
9     stop_id = row.get("stop_id").strip()
10    seq = int(row.get("stop_sequence").strip())
11
12    canon_stop = stop_to_canon.get(stop_id, stop_id)
13    trip_to_seq[trip_id].append((seq, canon_stop))

```

6.2.6 Étape 6 : chargement des géométries (shapes)

Si `shapes.txt` est présent, il est lu et les points sont triés selon `shape_pt_sequence`.

```

1 shape_points: dict[str, list[tuple[int, float, float]]] = defaultdict(list)
2
3 if has_shapes:
4     with shapes_path.open("r", encoding="utf-8-sig", newline="") as f:
5         r = csv.DictReader(f)
6         for row in r:
7             shape_id = row.get("shape_id").strip()
8             lat = float(row.get("shape_pt_lat").strip())
9             lon = float(row.get("shape_pt_lon").strip())
10            seq = int(row.get("shape_pt_sequence").strip())
11            shape_points[shape_id].append((seq, lon, lat))
12
13    for sid, pts in shape_points.items():
14        pts.sort(key=lambda x: x[0])

```

6.3 Regroupement des arrêts et déduplication

La canonicalisation des arrêts repose sur leurs coordonnées GPS. Elle garantit qu'un même arrêt physique n'apparaît qu'une seule fois dans le modèle final, même s'il est référencé par plusieurs identifiants dans les données source.

Cette approche présente plusieurs avantages :

- **Cohérence spatiale** : les arrêts situés au même endroit sont fusionnés, évitant les duplications visuelles sur la carte.
- **Simplification des requêtes** : les requêtes portant sur les arrêts deviennent plus simples et plus performantes.
- **Réduction de la taille des données** : le fichier JSON final contient moins d'arrêts redondants.

Lorsqu'un arrêt est utilisé par plusieurs lignes, il apparaît dans les données de chaque ligne concernée, mais conserve un identifiant canonique unique, facilitant les jointures et les analyses croisées.

6.4 Tri par « route la plus longue »

Pour chaque ligne, plusieurs trips peuvent exister. Afin de sélectionner un trajet représentatif et complet, une heuristique basée sur le nombre d'arrêts est appliquée : le trip comportant le plus grand nombre d'arrêts distincts est retenu comme référence.

```

1 for rid in route_ids:
2     trips = route_to_trips.get(rid, set())
3     best_trip_id = None
4     best_ordered: list[str] = []
5     best_len = -1
6
7     for tid in trips:
8         pairs = trip_to_seq.get(tid, [])
9         pairs_sorted = sorted(pairs, key=lambda x: x[0])
10
11         ordered: list[str] = []
12         last = None
13         for _, stop_canon in pairs_sorted:
14             if stop_canon != last:
15                 ordered.append(stop_canon)
16                 last = stop_canon
17
18         if len(ordered) > best_len:
19             best_len = len(ordered)
20             best_trip_id = tid
21             best_ordered = ordered

```

Cette méthode sélectionne automatiquement un parcours représentatif sans configuration manuelle. Elle pourrait être enrichie en intégrant un critère de distance géographique cumulée (par exemple via la formule de Haversine), ou la fréquence de desserte du trip.

6.5 Gestion des variantes et branches

Une même ligne peut présenter plusieurs variantes (trajets courts, détours temporaires, dessertes spécifiques). Le choix du trip « le plus long » capture généralement le parcours le plus complet. Les variantes non retenues peuvent être exploitées ultérieurement (affichage multi-itinéraires, analyses comparatives). Dans le cadre de ce projet, seule la variante principale est conservée afin de simplifier l’affichage et de garantir une meilleure lisibilité.

Il est également possible de distinguer explicitement les directions (aller/retour) en exploitant `direction_id`. Dans l’implémentation actuelle, cette distinction n’est pas appliquée : le trip sélectionné peut correspondre à l’une ou l’autre direction. Une amélioration consisterait à sélectionner deux trips distincts (un par direction) et à les représenter séparément ou via une structure unique contenant deux listes d’arrêts.

6.6 Production d’un GeoJSON final pour Mapbox

Une fois les arrêts et les géométries sélectionnés, les données sont structurées au format JSON. Deux fichiers principaux sont générés :

- **lines_normalized.json** : métadonnées de chaque ligne (identifiant, nom, couleur, liste d’arrêts, géométrie).
- **stops_normalized.json** : arrêts uniques avec coordonnées GPS et nom.

Exemple de format pour une ligne :

```

1 {
2     "id": 48,
3     "nom": "48",
4     "couleur": "FF0000",

```

```
5  "long_nom": "Rathburn",
6  "stopIds": [4501, 4502, 4503],
7  "geometries": [[lon1, lat1], [lon2, lat2]]
8  }
```

Exemple de format pour un arrêt :

```
1  {
2  "id": 4501,
3  "nom": "Rathburn at Islington",
4  "position": { "lat": 43.6529, "long": -79.5432 }
5  }
```

Ces structures sont ensuite converties côté frontend en **FeatureCollection** GeoJSON (LineString pour les lignes et Points pour les arrêts) afin d'être affichées par Mapbox GL.

6.7 Exposition via l'API REST

Le backend Node.js expose plusieurs endpoints pour accéder aux données des lignes et des arrêts.

GET /lines : retourne la liste des lignes disponibles.

```
1  router.get("/", async (_, res) => {
2    res.json(await getAllLines());
3  });
```

GET /lines/:id : retourne le détail complet d'une ligne.

```
1  router.get("/:id", async (req, res) => {
2    res.json(await getLineDetails(req.params.id));
3  });
```

POST /lines/bulk : retourne en une requête les détails de plusieurs arrêts.

```
1  router.post("/bulk", async (req, res) => {
2    res.json(await getStopsBulk(req.body.ids));
3  });
```

6.8 Choix techniques et limites du traitement GTFS

6.8.1 Choix techniques

- **Canonicalisation par coordonnées GPS** : efficace pour éliminer les doublons, mais dépend de la précision des coordonnées.
- **Sélection du trip le plus long** : pragmatique et robuste, mais ne prend pas en compte la fréquence ni la représentativité temporelle.
- **Utilisation de shapes.txt** : assure un tracé réaliste ; en son absence, un tracé approximatif par segments entre arrêts serait possible mais moins pertinent.

6.8.2 Limites

- **Qualité variable des données GTFS** : incohérences possibles (séquences erronées, arrêts manquants).
- **Shapes absentes/incomplètes** : limitation de la précision du tracé.
- **Gestion simplifiée des variantes** : un unique itinéraire principal est retenu.

- **Absence de gestion explicite des directions** : aller/retour non distingués dans l'implémentation actuelle.
- **Arrêts communs à plusieurs lignes** : information non exploitée directement dans l'interface.

6.9 Optimisations du pipeline GTFS

Le traitement GTFS est exécuté en amont, lors de la phase de préparation des données. Les résultats sont stockés sous forme de fichiers JSON lus par le backend au démarrage. Cette approche permet :

- des temps de réponse optimisés côté API ;
- une réduction de la charge CPU (pas de recalcul à la demande) ;
- une meilleure scalabilité, avec possibilité de servir ces fichiers via un CDN.

7 Amélioration du rendu et des performances

L'amélioration des performances de l'application cartographique a constitué un axe de travail majeur du projet. L'objectif n'était pas uniquement de rendre l'interface plus fluide, mais surtout de mettre en place une architecture plus cohérente et plus adaptée aux contraintes du rendu cartographique WebGL, tout en conservant les avantages offerts par React pour la gestion de l'interface et de l'état applicatif.

7.1 Analyse de la problématique initiale

Dans la version initiale de l'application, chaque ligne de bus ainsi que chaque arrêt étaient gérés de manière indépendante. Concrètement, cela se traduisait par la création d'une source et d'un layer Mapbox pour chaque ligne, et souvent d'un autre couple source/layer pour les arrêts associés. À cette structure s'ajoutait un composant React dédié par ligne, chargé d'instancier ces éléments graphiques.

Cette approche, bien que conceptuellement simple et proche du paradigme déclaratif de React, s'est révélée inadaptée aux contraintes de Mapbox GL. La multiplication des sources et des layers entraînait une surcharge importante du GPU et du CPU, provoquant des ralentissements notables lors du zoom, du déplacement de la carte ou encore lors de la sélection simultanée de plusieurs lignes.

De plus, la synchronisation fine entre le cycle de rendu de React et les opérations impératives de Mapbox (ajout et suppression de layers) augmentait la complexité du code et rendait l'application plus difficile à maintenir et à faire évoluer.

7.2 Principe de refonte retenu

Afin de corriger ces problèmes, une séparation plus nette des responsabilités entre React et Mapbox GL a été mise en place :

- React est utilisé pour la gestion de l'état applicatif, la logique métier et l'interface utilisateur.
- Mapbox GL est responsable exclusivement du rendu graphique et de l'affichage des données géographiques.

7.3 Mutualisation des sources et des layers

L'amélioration la plus significative concerne la mutualisation :

- une unique source GeoJSON regroupe l'ensemble des lignes de bus ;

- un seul layer `line` affiche toutes les lignes ;
- une seconde source GeoJSON regroupe l'ensemble des arrêts ;
- un unique layer `circle` affiche tous les arrêts.

7.4 Mise à jour des données via les sources existantes

Plutôt que de créer et détruire dynamiquement des layers à chaque interaction utilisateur, les mises à jour s'effectuent via `setData` sur les sources GeoJSON existantes. Lorsqu'un utilisateur sélectionne ou désélectionne une ligne :

- les données GeoJSON correspondantes sont reconstruites côté React ;
- les sources Mapbox existantes sont mises à jour ;
- Mapbox actualise automatiquement le rendu.

7.5 Gestion fiable et performante des sélections

La gestion des lignes sélectionnées repose sur des identifiants uniques stockés dans un `Set<string>`. Ce choix évite les problèmes de comparaisons par référence d'objets JavaScript et garantit des opérations rapides et fiables.

7.6 Allègement du cycle de rendu React

Les éléments visuels lourds (lignes et arrêts) ne sont plus liés au cycle de rendu des composants React. React se limite à mettre à jour l'état et déclencher les reconstructions de données, tandis que Mapbox GL gère l'affichage.

7.7 Résultats et bénéfices observés

La nouvelle architecture permet :

- une fluidité accrue lors de la navigation sur la carte ;
- une réduction significative de la charge GPU et CPU ;
- une meilleure lisibilité et maintenabilité du code ;
- une base solide pour des optimisations futures (LOD, simplification géométrique, chargement à la demande).

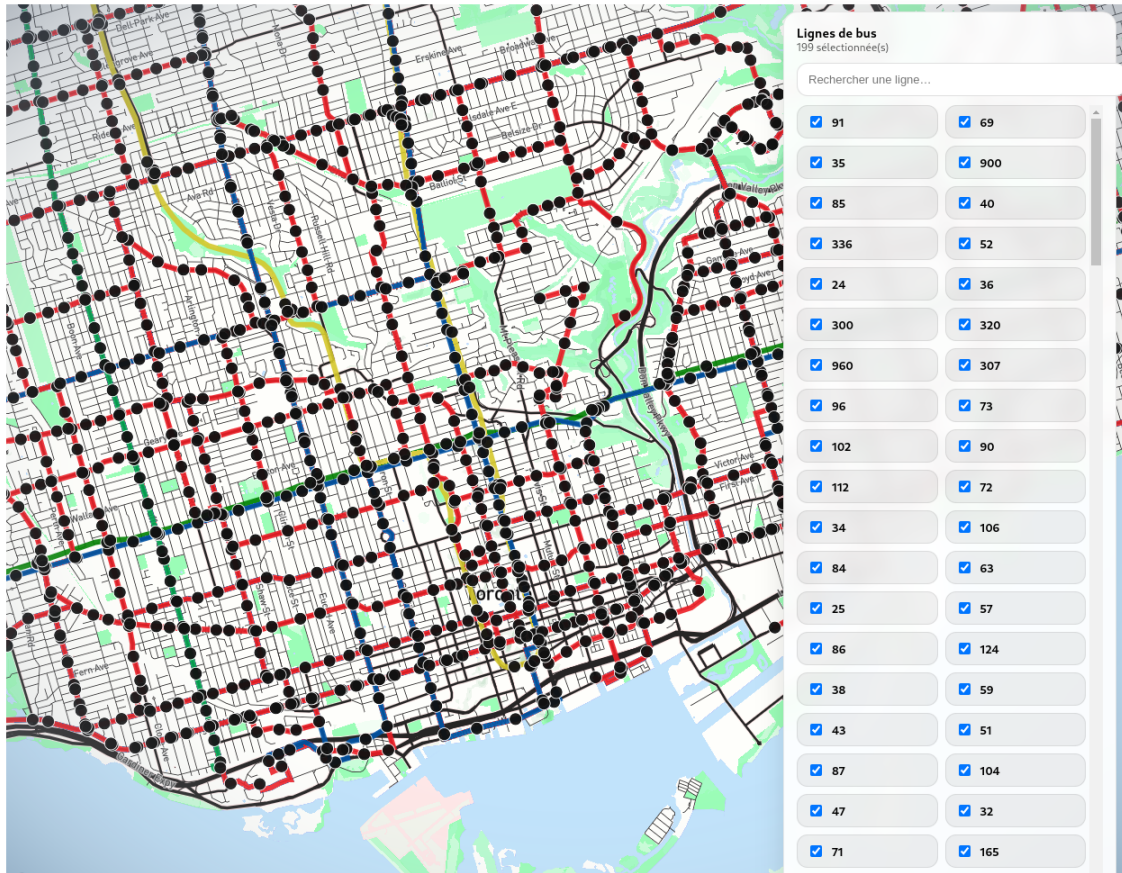


FIGURE 4 – Amélioration du rendu : affichage simultané des lignes

8 Intégration du service de prédiction des retards

L'un des objectifs majeurs du projet était de permettre aux usagers d'anticiper les perturbations sur le réseau de transport. Pour répondre à cette problématique, un système de prédiction des retards a été développé et intégré à l'application.

8.1 Architecture du système de prédiction

Le système repose sur une architecture en trois couches :

1. **Modèle de Machine Learning** : un modèle XGBoost entraîné sur des données historiques combinant informations météorologiques, temporelles et incidents passés.
2. **API de prédiction Python (Flask)** : un service REST exposant le modèle via des endpoints HTTP afin d'obtenir des prédictions en temps réel.
3. **Intégration Front-end et Back-end** : une couche TypeScript orchestrant les appels au service de prédiction et présentant les résultats à l'utilisateur.

8.2 Modèle de prédiction simplifié

Le modèle a été optimisé pour concilier performance et simplicité :

- **Encodage cyclique** des variables temporelles (heure, jour de la semaine, mois, direction du vent).
- **Transformation des variables météorologiques** : regroupement des conditions similaires, binarisation des précipitations, catégorisation de la visibilité.
- **Prétraitement standardisé** via un pipeline scikit-learn sauvegardé.

8.3 API de prédiction

L'API Flask (`app.py`) expose :

- **POST /predict** : accepte des données météorologiques et temporelles et retourne une estimation du retard en minutes.

L'API utilise CORS pour permettre les appels depuis le front-end et inclut une validation des données d'entrée afin de garantir la fiabilité des prédictions.

8.4 Intégration Backend (TypeScript/Node.js)

Le backend TypeScript fait office d'intermédiaire entre le front-end et le service Python :

- **predictionService.ts** : communication avec l'API Flask (gestion d'erreurs et vérification de santé).
- **busRoutes.ts** : endpoint `POST /lines/:id/prediction/:incident`.
- **busService.ts** : intégration des appels de prédiction dans la logique métier.

L'URL de l'API de prédiction est configurable via `PREDICTION_API_URL`.

8.5 Intégration Frontend (React)

Les prédictions sont intégrées dans le panneau de statistiques :

- **PredictionService.tsx** : encapsule les appels API vers le backend.
- **StatsPanel.tsx** : charge les prédictions lors de la sélection d'une ligne et affiche le retard prédit.
- **StatsPanel.module.css** : section dédiée pour l'affichage (badge, bordures arrondies).

8.6 Flux de données

1. L'utilisateur sélectionne une ou plusieurs lignes de bus sur la carte.
2. Le frontend récupère les données météorologiques actuelles.
3. Pour chaque ligne sélectionnée, un appel API est effectué vers le backend.
4. Le backend transmet la requête au service Python de prédiction.
5. Le modèle XGBoost génère une prédiction.
6. Le résultat remonte jusqu'au frontend.
7. Les prédictions sont affichées dans le panneau de statistiques.

9 Guide d'utilisation

9.1 Démarrage de l'application complète

Pour bénéficier des prédictions de retard, l'application nécessite le démarrage de trois services.

1. Service de prédiction Python

```
cd models
pip install -r requirements.txt
python app.py
```

Le service démarre par défaut sur `http://localhost:5000`.

2. Backend Node.js

```
cd app/backend
npm install
npm start
```

Le backend démarre sur le port 3000 et communique avec le service de prédiction.

3. Frontend React

```
cd app/frontend
npm install
npm run dev
```

L'interface utilisateur est accessible sur `http://localhost:5173`, à condition de spécifier les tokens nécessaires dans les fichiers `.env` de chaque module :

- Frontend : `VITE_MAPBOX_ACCESS_TOKEN` et `VITE_API_BASE_URL`
- Backend : `MAPBOX_ACCESS_TOKEN`, `PORT`, `PREDICTION_API_URL`, `BASLINE_LINE_FOR_PREDICTIONS`, `WEATHER_API_URL`, `WEATHER_API_KEY`

9.2 Utilisation de la fonctionnalité de prédiction

1. Ouvrez l'application dans votre navigateur.
2. Sur la carte interactive, sélectionnez une ou plusieurs lignes de bus en utilisant le panneau latéral gauche.
3. Le panneau de statistiques à droite affiche automatiquement :
 - les statistiques globales du réseau ;
 - les prédictions par type d'incident ;
 - **les retards prédits en minutes pour chaque ligne sélectionnée.**

Les prédictions sont calculées en temps réel en tenant compte des conditions météorologiques actuelles et de l'heure de la journée.

10 Intégration du service de données météorologiques

Pour améliorer la précision et la fiabilité du système de prédiction, un service majeur a été développé :

- **Service de données météorologiques** : fournit des données météorologiques réelles au système prédictif.

10.1 Architecture du service météorologique

Le service météorologique repose sur l'API : <https://www.meteosource.com/>

Backend (weatherService.ts) :

- récupère les données actuelles à Toronto via l'API;
- implémente un système de cache pour optimiser les performances (durée : 1 heure);
- expose le format de données spécifié par le système prédictif.

API REST :

- GET /weather/current : récupère les conditions météorologiques actuelles.

Frontend (WeatherService.tsx) : encapsule les appels API et fournit des méthodes pour récupérer les données météorologiques.

```

1 export type WeatherData = {
2   TEMP: number;
3   DEW_POINT_TEMP: number;
4   HUMIDEX: number | null;
5   PRECIP_AMOUNT: number;
6   RELATIVE_HUMIDITY: number;
7   STATION_PRESSURE: number;
8   VISIBILITY: number;
9   WEATHER_ENG_DESC: string;
10  WIND_DIRECTION: number | null;
11  WIND_SPEED: number | null;
12  LOCAL_DATE: string;
13  LOCAL_TIME: string;
14  WEEK_DAY: string;
15  LOCAL_MONTH: number;
16  LOCAL_DAY: number;
17 }

```

10.2 Intégration dans le pipeline de prédiction

Le service météorologique s'intègre étroitement au système de prédiction. À chaque requête de prédiction pour une ligne donnée, le backend récupère automatiquement les conditions météorologiques via /weather/current et transmet ces données au service Python.

Le format produit par `weatherService.ts` correspond aux exigences du modèle XGBoost, garantissant la compatibilité et évitant des conversions intermédiaires. Le système de cache (1 heure) limite le nombre d'appels à l'API externe tout en conservant une fraîcheur raisonnable des données.

De plus nous récupérerons la description du temps pour afficher sur la carte dynamiquement l'état **neige** et **pluie** (voir POC) :

```

1   if(condition.includes("rain")){
2       // @ts-ignore
3       map.setRain({
4         density: zoomBasedReveal(0.2),
5         intensity: 1.0,
6         color: '#a8adbc',
7         opacity: 0.4,
8         direction: [0, 80],
9         'droplet-size': [2.6, 18.2],
10        'distortion-strength': 0.7,
11        'center-thinning': 0

```

```
12     })
13   }else if(condition.includes("snow")){
14     // @ts-ignore
15     map.setSnow({
16       density: zoomBasedReveal(0.4),
17       intensity: 1.0,
18       'center-thinning': 0.1,
19       direction: [0, 50],
20       opacity: 1.0,
21       color: '#ffffff',
22       'flake-size': 0.71,
23       vignette: zoomBasedReveal(0.3),
24       'vignette-color': '#ffffff'
25     });
26   }
```

11 Limites et perspectives

Bien que l'application ait atteint un niveau fonctionnel satisfaisant et remplisse les objectifs principaux fixés, plusieurs limitations ont été identifiées au cours du développement, ouvrant la voie à des améliorations futures.

11.1 Limitations rencontrées

11.1.1 Impossibilité d'afficher les bus en 3D avec Three.js

Une fonctionnalité envisagée consistait à intégrer une visualisation tridimensionnelle des bus en temps réel. Cette fonctionnalité s'est heurtée à une contrainte technique majeure : les API publiques disponibles pour le réseau TTC de Toronto ne fournissent pas la position géographique en temps réel des véhicules, mais uniquement des informations de type *temps estimé avant le prochain passage*.

Sans coordonnées GPS, il n'est pas possible de représenter les véhicules sur la carte. Une alternative consisterait à simuler les positions par interpolation, mais produirait des données approximatives.

Perspectives : si l'API TTC évolue vers un flux GTFS-RT exposant des positions GPS, une visualisation 2D/3D en temps réel pourrait être intégrée. Une architecture WebSocket permettrait de diffuser les positions en continu.

11.1.2 Absence d'étude statistique historique

Une analyse approfondie des données historiques (corrélations météo/incidents, tendances temporelles) n'a pas été développée par manque de temps.

Perspectives : la mise en place d'une base de données (PostgreSQL/MongoDB) permettrait de stocker incidents, prédictions et météo au fil du temps, et de proposer des tableaux de bord interactifs (Chart.js/D3.js).

11.2 Limitations techniques et données

11.2.1 Qualité et complétude des données GTFS

Certaines lignes ne disposent pas de `shapes.txt`, limitant la précision des tracés. Des incohérences d'arrêts (doublons, coordonnées imprécises) nécessitent des traitements correctifs.

Perspectives : correction manuelle ou semi-automatique et reconstruction de géométries via une API de routage (ex. Mapbox Directions utilisé lors du POC), au prix d'un coût API.

11.2.2 Gestion simplifiée des variantes et directions

La version actuelle ne distingue pas explicitement aller/retour et variantes horaires.

Perspectives : exploiter `direction_id` pour afficher les deux directions, et proposer plusieurs itinéraires représentatifs via un sélecteur.

11.2.3 Absence de temps réel pour les prédictions

Les prédictions reposent sur des données historiques et la météo actuelle, sans état réel du trafic.

Perspectives : intégration de données de trafic en temps réel.

11.3 Perspectives d'extension

11.3.1 Internationalisation et support multi-villes

L'application est conçue pour le réseau TTC de Toronto.

Perspectives : généralisation multi-villes via configuration et sélection de réseau dans l'interface, en consommant différents flux GTFS.

11.3.2 Optimisations pour les très grands réseaux

Pour des réseaux de grande taille, des optimisations supplémentaires seraient nécessaires.

Perspectives : clustering spatial des arrêts à faible zoom, chargement progressif des données (lazy loading), simplification géométrique dynamique et adaptation du niveau de détail.

12 Conclusion

Ce projet a permis de concevoir et de développer une application web complète de visualisation et d'analyse du réseau de transport urbain de Toronto, combinant plusieurs technologies modernes et offrant une expérience utilisateur riche et interactive.

12.1 Bilan des objectifs atteints

Les objectifs fonctionnels prioritaires ont été atteints :

- **Carte interactive** : intégration de Mapbox GL JS avec React, affichage des lignes et des arrêts, et amélioration notable des performances grâce à la mutualisation des sources et layers.
- **Prédiction des retards** : modèle XGBoost exposé via Flask et intégré à l'application via une orchestration backend TypeScript.
- **Statistiques réseau et par ligne** : affichage d'indicateurs globaux et par ligne pour faciliter l'analyse par les usagers.
- **Météo actuelle** : intégration de Meteosource avec cache afin de limiter les appels externes.

12.2 Apports du projet

Sur le plan technique, ce projet a permis de mettre en œuvre :

- un pipeline complet de traitement et d'exploitation des données GTFS (lecture, nettoyage, canonicalisation et structuration) ;

- une architecture modulaire en services (frontend, backend, microservice de prédiction) ;
- des optimisations de rendu cartographique respectant les contraintes WebGL ;
- l'intégration d'un modèle de Machine Learning dans une application web via une API dédiée.

12.3 Perspectives d'évolution

Les limitations identifiées ouvrent plusieurs axes d'amélioration :

- accès aux positions GPS temps réel pour une visualisation dynamique des bus ;
- stockage et analyse des données historiques pour des statistiques avancées ;
- généralisation multi-villes et amélioration de la gestion des variantes/directions.

En conclusion, ce projet constitue une base fonctionnelle et techniquement solide, démontrant la faisabilité d'une plateforme complète de visualisation et d'analyse du transport urbain.